

Laxmi Charitable Trust's
Sheth L.U.J College of Arts & Sir M.V. College of Science and Commerce
Department of Information Technology (B.Sc.I.T Semester V)
APPLIED BUSINESS ANALYTICS
Practical –XI

Roll No.:T006	Name:Nandini Chaudhari
Class:TYIT	Batch:01
Date of Assignment:03-10-2026	Date/Time of Submission:03-10-2026

Practical 11: Optimization Modeling and Simulation Modeling

Part A: Optimization Modeling-Business Problem: A company produces two products, Product A and Product B. The

The company wants to maximize its profit using the available machine and labour hours.-Available Machine Hours = 100, Available Labour Hours = 80

Practical Questions:

```
from scipy.optimize import linprog
```

1. Define the decision variables for Product A and Product B.

```
# Decision Variables
x = 0 # Number of units of Product A
y = 0 # Number of units of Product B
print("Decision Variable x = Units of Product A")
print("Decision Variable y = Units of Product B")
```

```
... Decision Variable x = Units of Product A
Decision Variable y = Units of Product B
```

2. Define the objective of the problem.

```
# Objective
# Profit per unit:
profit_A = 40
profit_B = 30
print("Objective: Maximize Total Profit")
print("Profit per unit of Product A =", profit_A)
print("Profit per unit of Product B =", profit_B)
```

```
... Objective: Maximize Total Profit
Profit per unit of Product A = 40
Profit per unit of Product B = 30
```

3. Formulate the objective function to maximize the total profit.

```
# Objective Function
# Maximize:
# Z = 40x + 30y
def calculate_profit(x, y):
    return 40*x + 30*y
print("Objective Function:")
print("Maximize Z = 40x + 30y")
```

```
... Objective Function:
Maximize Z = 40x + 30y
```

4. Formulate the machine-hour constraint.

```
# Machine Hour Constraint
print("Machine Hour Constraint:")
print("2x + y <= 100")
```

```
... Machine Hour Constraint:
2x + y <= 100
```

5. Formulate the labour-hour constraint.

```
# Labour Hour Constraint
print("Labour Hour Constraint:")
print("x + 2y <= 80")
```

```
... Labour Hour Constraint:
x + 2y <= 80
```

6. Add non-negative constraints for the decision variables.

```
# Non-negative Constraints
print("Non-negative Constraints:")
print("x >= 0")
print("y >= 0")
```

```
Non-negative Constraints:
x >= 0
y >= 0
```

7. Solve the linear programming problem using Python.

```
from scipy.optimize import linprog

# Objective function coefficients
# linprog performs minimization, so use negative values for maximization
c = [-40, -30]

# Constraints
A = [
    [2, 1], # Machine hours
    [1, 2] # Labour hours
]
b = [100, 80]

# Bounds for x and y
bounds = [(0, None), (0, None)]

# Solve Linear Programming Problem
result = linprog(c, A_ub=A, b_ub=b, bounds=bounds, method='highs')

print(result)
```

```
message: Optimization terminated successfully. (HiGHS Status 7: Optimal)
success: True
status: 0
fun: -2200.0
x: [ 4.000e+01  2.000e+01]
nit: 2
lower: residual: [ 4.000e+01  2.000e+01]
marginals: [ 0.000e+00  0.000e+00]
upper: residual: [          inf          inf]
marginals: [ 0.000e+00  0.000e+00]
eqlin: residual: []
marginals: []
ineqlin: residual: [ 0.000e+00  0.000e+00]
marginals: [-1.667e+01 -6.667e+00]
mip_node_count: 0
mip_dual_bound: 0.0
mip_gap: 0.0
```

8. Find the optimal number of units of Product A and Product B.

```

# Optimal Solution

if result.success:
    x_opt = result.x[0]
    y_opt = result.x[1]

    print("Optimal units of Product A:", x_opt)
    print("Optimal units of Product B:", y_opt)
else:
    print("No optimal solution found.")

```

```

... Optimal units of Product A: 40.0
    Optimal units of Product B: 20.0

```

9. Calculate the maximum profit.

```

# Maximum Profit

if result.success:
    maximum_profit = 40*x_opt + 30*y_opt

    print("Maximum Profit =", maximum_profit)

```

```

... Maximum Profit = 2200.0

```

10. Check whether the available machine and labour hours are satisfied.

```

# Check Resource Constraints
machine_used = 2*x_opt + y_opt
labour_used = x_opt + 2*y_opt
print("Machine hours used:", machine_used)
print("Machine hours available:", 100)
print("Labour hours used:", labour_used)
print("Labour hours available:", 80)
if machine_used <= 100:
    print("Machine hour constraint satisfied.")
else:
    print("Machine hour constraint violated.")
if labour_used <= 80:
    print("Labour hour constraint satisfied.")
else:
    print("Labour hour constraint violated.")

```

```

... Machine hours used: 100.0
    Machine hours available: 100
    Labour hours used: 80.0
    Labour hours available: 80
    Machine hour constraint satisfied.
    Labour hour constraint satisfied.

```

11. Display the optimal solution in a suitable format.

```

import pandas as pd

optimal_solution = pd.DataFrame({
    "Product": ["Product A", "Product B"],
    "Optimal Units": [x_opt, y_opt],
    "Profit per Unit": [40, 30]
})

optimal_solution["Total Profit"] = (
    optimal_solution["Optimal Units"] *
    optimal_solution["Profit per Unit"]
)

print(optimal_solution)

```

```

...
   Product  Optimal Units  Profit per Unit  Total Profit
0  Product A           40.0             40          1600.0
1  Product B           20.0             30           600.0

```

12. Interpret the optimal solution in a business context.

```

print("Business Interpretation:")
print(
    f"The company should produce {x_opt:.0f} units of Product A "
    f"and {y_opt:.0f} units of Product B."
)

print(
    f"This production plan generates a maximum profit of "
    f"Rs. {maximum_profit:.0f}."
)

print(
    "The available machine and labour hours are fully utilized."
)

```

```

Business Interpretation:
The company should produce 40 units of Product A and 20 units of Product B.
This production plan generates a maximum profit of Rs. 2200.
The available machine and labour hours are fully utilized.

```

13. Write a conclusion based on the optimization result.

Conclusion:

The linear programming model determines the optimal production plan for maximizing profit. The company should produce 40 units of Product A and 20 units of Product B, resulting in a maximum profit of Rs. 2200. Both machine and labour constraints are completely satisfied.

Part B: Simulation Modeling-Business Problem: A supermarket wants to decide the appropriate inventory level for a product when daily demand is uncertain.

Practical Questions:

14. Generate random daily demand values for a product for 30 days.

```

import numpy as np
import pandas as pd

# Set seed so that results remain reproducible
np.random.seed(42)

# Generate demand for 30 days
demand = np.random.randint(10, 51, size=30)

print("Daily Demand for 30 Days:")
print(demand)

```

```

Daily Demand for 30 Days:
[48 38 24 17 30 48 28 32 20 20 33 45 49 33 12 31 11 33 39 47 11 30 42 21
 31 34 36 37 25 24]

```

15. Display the generated demand values.

```
Output exceeds the size limit. Open
Day 1: Demand = 48 units
Day 2: Demand = 38 units
Day 3: Demand = 24 units
Day 4: Demand = 17 units
Day 5: Demand = 30 units
Day 6: Demand = 48 units
Day 7: Demand = 28 units
Day 8: Demand = 32 units
Day 9: Demand = 20 units
Day 10: Demand = 20 units
Day 11: Demand = 33 units
Day 12: Demand = 45 units
Day 13: Demand = 49 units
Day 14: Demand = 33 units
Day 15: Demand = 12 units
Day 16: Demand = 31 units
Day 17: Demand = 11 units
Day 18: Demand = 33 units
Day 19: Demand = 39 units
Day 20: Demand = 47 units
Day 21: Demand = 11 units
Day 22: Demand = 30 units
Day 23: Demand = 42 units
Day 24: Demand = 21 units
Day 25: Demand = 31 units
...
Day 27: Demand = 36 units
Day 28: Demand = 37 units
Day 29: Demand = 25 units
Day 30: Demand = 24 units
```

```
# Display demand values

for day, value in enumerate(demand, start=1):
    print(f"Day {day}: Demand = {value} units")

✓ 0.0s
```

16. Select an inventory level of 30 units.

```
# Inventory Level

inventory = 30

print("Selected Inventory Level:", inventory, "units")

✓ 0.0s
```

Selected Inventory Level: 30 units

17. Compare the daily demand with the available inventory.

```
# Compare demand with inventory

for day, daily_demand in enumerate(demand, start=1):

    if daily_demand <= inventory:
        print(
            f"Day {day}: Demand = {daily_demand}, "
            f"Inventory = {inventory} -> Sufficient"
        )
    else:
        print(
            f"Day {day}: Demand = {daily_demand}, "
            f"Inventory = {inventory} -> Shortage"
        )

✓ 0.0s
```

```
Output exceeds the size limit. Open the full output data in a text editor
Day 1: Demand = 48, Inventory = 30 -> Shortage
Day 2: Demand = 38, Inventory = 30 -> Shortage
Day 3: Demand = 24, Inventory = 30 -> Sufficient
Day 4: Demand = 17, Inventory = 30 -> Sufficient
Day 5: Demand = 30, Inventory = 30 -> Sufficient
Day 6: Demand = 48, Inventory = 30 -> Shortage
Day 7: Demand = 28, Inventory = 30 -> Sufficient
Day 8: Demand = 32, Inventory = 30 -> Shortage
Day 9: Demand = 20, Inventory = 30 -> Sufficient
Day 10: Demand = 20, Inventory = 30 -> Sufficient
Day 11: Demand = 33, Inventory = 30 -> Shortage
Day 12: Demand = 45, Inventory = 30 -> Shortage
Day 13: Demand = 49, Inventory = 30 -> Shortage
Day 14: Demand = 33, Inventory = 30 -> Shortage
Day 15: Demand = 12, Inventory = 30 -> Sufficient
Day 16: Demand = 31, Inventory = 30 -> Shortage
Day 17: Demand = 11, Inventory = 30 -> Sufficient
Day 18: Demand = 33, Inventory = 30 -> Shortage
Day 19: Demand = 39, Inventory = 30 -> Shortage
Day 20: Demand = 47, Inventory = 30 -> Shortage
Day 21: Demand = 11, Inventory = 30 -> Sufficient
Day 22: Demand = 30, Inventory = 30 -> Sufficient
Day 23: Demand = 42, Inventory = 30 -> Shortage
Day 24: Demand = 21, Inventory = 30 -> Sufficient
Day 25: Demand = 31, Inventory = 30 -> Shortage
...
Day 27: Demand = 36, Inventory = 30 -> Shortage
Day 28: Demand = 37, Inventory = 30 -> Shortage
Day 29: Demand = 25, Inventory = 30 -> Sufficient
Day 30: Demand = 24, Inventory = 30 -> Sufficient
```

18. Calculate the number of units sold each day.

```
Output exceeds the size limit. Open the full output data in a text editor
Day 1: Demand = 48, Units Sold = 30
Day 2: Demand = 38, Units Sold = 30
Day 3: Demand = 24, Units Sold = 24
Day 4: Demand = 17, Units Sold = 17
Day 5: Demand = 30, Units Sold = 30
Day 6: Demand = 48, Units Sold = 30
Day 7: Demand = 28, Units Sold = 30
Day 8: Demand = 32, Units Sold = 30
Day 9: Demand = 20, Units Sold = 20
Day 10: Demand = 20, Units Sold = 20
Day 11: Demand = 33, Units Sold = 30
Day 12: Demand = 45, Units Sold = 30
Day 13: Demand = 49, Units Sold = 30
Day 14: Demand = 33, Units Sold = 30
Day 15: Demand = 12, Units Sold = 12
Day 16: Demand = 31, Units Sold = 30
Day 17: Demand = 11, Units Sold = 11
Day 18: Demand = 33, Units Sold = 30
Day 19: Demand = 39, Units Sold = 30
Day 20: Demand = 47, Units Sold = 30
Day 21: Demand = 11, Units Sold = 11
Day 22: Demand = 30, Units Sold = 30
Day 23: Demand = 42, Units Sold = 30
Day 24: Demand = 21, Units Sold = 21
Day 25: Demand = 31, Units Sold = 30
...
Day 27: Demand = 36, Units Sold = 30
Day 28: Demand = 37, Units Sold = 30
Day 29: Demand = 25, Units Sold = 25
Day 30: Demand = 24, Units Sold = 24
```

```
# Calculate units sold

units_sold = np.minimum(demand, inventory)

for day in range(30):
    print(
        f"Day {day+1}: Demand = {demand[day]}, "
        f"Units Sold = {units_sold[day]}"
    )

✓ 0.0s
```

19. Calculate the number of units left after each day.

```
Output exceeds the size limit. Open the full output data in a text editor
Day 1: Units Sold = 30, Units Left = 0
Day 2: Units Sold = 30, Units Left = 0
Day 3: Units Sold = 24, Units Left = 6
Day 4: Units Sold = 17, Units Left = 13
Day 5: Units Sold = 30, Units Left = 0
Day 6: Units Sold = 30, Units Left = 0
Day 7: Units Sold = 28, Units Left = 2
Day 8: Units Sold = 30, Units Left = 0
Day 9: Units Sold = 20, Units Left = 10
Day 10: Units Sold = 20, Units Left = 10
Day 11: Units Sold = 30, Units Left = 0
Day 12: Units Sold = 30, Units Left = 0
Day 13: Units Sold = 30, Units Left = 0
Day 14: Units Sold = 30, Units Left = 0
Day 15: Units Sold = 12, Units Left = 18
Day 16: Units Sold = 30, Units Left = 0
Day 17: Units Sold = 11, Units Left = 19
Day 18: Units Sold = 30, Units Left = 0
Day 19: Units Sold = 30, Units Left = 0
Day 20: Units Sold = 30, Units Left = 0
Day 21: Units Sold = 11, Units Left = 19
Day 22: Units Sold = 30, Units Left = 0
Day 23: Units Sold = 30, Units Left = 0
Day 24: Units Sold = 21, Units Left = 9
Day 25: Units Sold = 30, Units Left = 0
...
Day 27: Units Sold = 30, Units Left = 0
Day 28: Units Sold = 30, Units Left = 0
Day 29: Units Sold = 25, Units Left = 5
Day 30: Units Sold = 24, Units Left = 6
```

```
# Calculate remaining inventory

inventory_left = inventory - units_sold

for day in range(30):
    print(
        f"Day {day+1}: "
        f"Units Sold = {units_sold[day]}, "
        f"Units Left = {inventory_left[day]}"
    )

✓ 0.0s
```

20. Calculate the shortage when demand is greater than available inventory.

Output exceeds the [size limit](#). Open the full output data [in a text editor](#)

```

Day 1: Demand = 48, Shortage = 18
Day 2: Demand = 38, Shortage = 8
Day 3: Demand = 24, Shortage = 0
Day 4: Demand = 17, Shortage = 0
Day 5: Demand = 30, Shortage = 0
Day 6: Demand = 48, Shortage = 18
Day 7: Demand = 28, Shortage = 0
Day 8: Demand = 32, Shortage = 2
Day 9: Demand = 20, Shortage = 0
Day 10: Demand = 20, Shortage = 0
Day 11: Demand = 33, Shortage = 3
Day 12: Demand = 45, Shortage = 15
Day 13: Demand = 49, Shortage = 19
Day 14: Demand = 33, Shortage = 3
Day 15: Demand = 12, Shortage = 0
Day 16: Demand = 31, Shortage = 1
Day 17: Demand = 11, Shortage = 0
Day 18: Demand = 33, Shortage = 3
Day 19: Demand = 39, Shortage = 9
Day 20: Demand = 47, Shortage = 17
Day 21: Demand = 11, Shortage = 0
Day 22: Demand = 30, Shortage = 0
Day 23: Demand = 42, Shortage = 12
Day 24: Demand = 21, Shortage = 0
Day 25: Demand = 31, Shortage = 1
...
Day 27: Demand = 36, Shortage = 6
Day 28: Demand = 37, Shortage = 7
Day 29: Demand = 25, Shortage = 0
Day 30: Demand = 24, Shortage = 0

```

```

# Calculate shortage
shortage = np.maximum(demand - inventory, 0)

for day in range(30):
    print(
        f"Day {day+1}: "
        f"Demand = {demand[day]}, "
        f"Shortage = {shortage[day]}"
    )

```

✓ 0.0s

21. Calculate the total units sold and total shortage for 30 days.

```

# Total units sold
total_units_sold = units_sold.sum()

# Total shortage
total_shortage = shortage.sum()

print("Total Units Sold:", total_units_sold)
print("Total Shortage:", total_shortage)

```

✓ 0.0s

Total Units Sold: 783
Total Shortage: 146

22. Repeat the simulation using inventory levels of 30, 40, and 50 units.

```

# Inventory levels
inventory_levels = [30, 40, 50]

for inventory in inventory_levels:
    units_sold = np.minimum(demand, inventory)
    shortage = np.maximum(demand - inventory, 0)

    total_sales = units_sold.sum()
    total_shortage = shortage.sum()

    print("\nInventory Level:", inventory)
    print("Total Units Sold:", total_sales)
    print("Total Shortage:", total_shortage)

```

✓ 0.0s

Inventory Level: 30
Total Units Sold: 783
Total Shortage: 146

Inventory Level: 40
Total Units Sold: 890
Total Shortage: 39

Inventory Level: 50
Total Units Sold: 929
Total Shortage: 0

23. Store the results of all three scenarios in a DataFrame.

```

# Store simulation results
results = []

for inventory in inventory_levels:
    units_sold = np.minimum(demand, inventory)
    shortage = np.maximum(demand - inventory, 0)

    results.append({
        "Inventory Level": inventory,
        "Total Units Sold": units_sold.sum(),
        "Total Shortage": shortage.sum()
    })

results_df = pd.DataFrame(results)

print(results_df)

```

✓ 0.0s

	Inventory Level	Total Units Sold	Total Shortage
0	30	783	146
1	40	890	39
2	50	929	0

24. Compare the total sales and total shortage for each inventory level.

```

# Compare scenarios
print("Comparison of Inventory Scenarios:\n")

for index, row in results_df.iterrows():
    print(
        f"Inventory = {row['Inventory Level']} units | "
        f"Total Sales = {row['Total Units Sold']} units | "
        f"Total Shortage = {row['Total Shortage']} units"
    )

```

✓ 0.0s

Comparison of Inventory Scenarios:

Inventory = 30 units | Total Sales = 783 units | Total Shortage = 146 units
 Inventory = 40 units | Total Sales = 890 units | Total Shortage = 39 units
 Inventory = 50 units | Total Sales = 929 units | Total Shortage = 0 units

25. Create a chart to compare the results of the three scenarios.

```

import matplotlib.pyplot as plt

# Plot Total Sales
plt.figure(figsize=(8, 5))

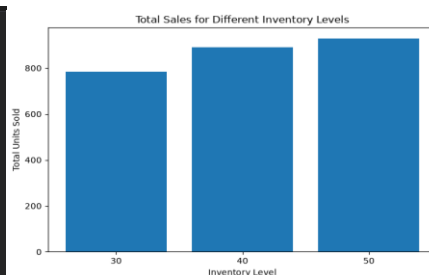
plt.bar(
    results_df["Inventory Level"].astype(str),
    results_df["Total Units Sold"]
)

plt.xlabel("Inventory Level")
plt.ylabel("Total Units Sold")
plt.title("Total Sales for Different Inventory Levels")

plt.show()

```

✓ 0.3s



26. Identify how changing the inventory level affects sales and shortage.

```
print("Effect of Inventory Level:\n")
for index, row in results_df.iterrows():
    print(
        f"Inventory Level {row['Inventory Level']} units: "
        f"Sales = {row['Total Units Sold']} units, "
        f"Shortage = {row['Total Shortage']} units"
    )

print("\nObservation:")
print(
    "As the inventory level increases, the total shortage generally decreases "
    "because more products are available to satisfy customer demand."
)

print(
    "Total sales generally increase until the inventory level is sufficient "
    "to satisfy the generated demand."
)
```

Effect of Inventory Level:

Inventory Level 30 units: Sales = 783 units, Shortage = 146 units
Inventory Level 40 units: Sales = 890 units, Shortage = 39 units
Inventory Level 50 units: Sales = 929 units, Shortage = 0 units

Observation:

As the inventory level increases, the total shortage generally decreases because more products are available to satisfy customer demand.
Total sales generally increase until the inventory level is sufficient to satisfy the generated demand.

27. Write a conclusion about the inventory level based on the simulation

Conclusion:

Based on the 30-day simulation, an inventory level of 50 units resulted in the lowest shortage of 0 units.

Increasing inventory reduces the risk of stock shortages, but it may also result in more unsold inventory.

Therefore, the business should select an inventory level by balancing expected demand and shortage risk.